

Ambigram-Anamorph Module (AmbiAnamorph)

One-page concise spec — an input-agnostic, lean, encapsulated module for combining intrinsic 3D ambigram geometry with anamorphic projection/warps for real-time and offline pipelines. Designed so a developer can implement the module from this spec.

1. Overview & goals

- **Purpose:** Provide a reusable library that (a) constructs or ingests geometry/textures that are *intrinsically ambigrammatic* in 3D, (b) computes/apply anamorphic warps driven by camera/view parameters, and (c) exposes optimization, rendering, and export APIs.
- **Design constraints:** Input-agnostic (accepts raster images, SDF/MSDF, vector outlines, parametric curves, or 3D meshes). Read/write enabled (load and export meshes, warp maps, lookup textures). Minimal dependencies and clear separation between offline optimizers and real-time runtime.

2. Core concepts

- **Intrinsic ambigram:** a 3D shape whose projection(s) from specified viewpoints match target glyphs/silhouettes.
- **Anamorphism:** a projective/cylindrical mapping applied to rendered frames or textures so they resolve correctly for a target viewpoint.
- **View manifold S:** the set of viewer parameters (position/orientation/projection) that satisfy a legibility predicate for a target reading. The module provides tools to sample, map and visualize S.

3. Architecture (high level)

Components (clean separation): 1. **IO Layer** — load/save images, SDFs, meshes (OBJ/GLTF/STL), param curves (SVG/GeoJSON), and write outputs. 2. **Preprocessor** — normalizes inputs to internal formats (SDF for 2D strokes; mesh or control-mesh for 3D). 3. **Representation** — parametric mesh/SDF/grid abstraction used by optimizer and renderer. (Simple implementations: vertex mesh + stroke SDF textures.) 4. **Optimizer** — enforces multi-view silhouette constraints and geometric regularizers (differentiable or evolutionary backend pluggable). 5. **Projector / Warper** — compute homographies (planar), cylindrical/conical reflect mappings, or arbitrary 3D→2D LUTs and produce inverse lookup textures for shaders. 6. **Renderer (runtime)** — Three.js/WebGL target for preview & realtime. Accepts optimized geometry and warp maps. Exports render targets. 7. **Validator / Sampler** — samples camera space V, computes legibility metric (mask IoU, SDF L2, OCR score) and returns S topology (connected components, dims). 8. **Utilities** — SDF/MSDF tools, smoothing, topology checks, manufacturability constraints (min thickness), export helpers.

4. Public API (class-style)

```
// Constructor
const ambi = new AmbiAnamorph(config)
```

```

// Key methods
await ambi.loadSource(source)           // source: {type:
'image'|'sdf'|'svg'|'mesh', data: ...}
ambi.setTargets([id, imageOrSDF, tolerance])
ambi.setViewpoints([id, cameraMatrix, projection, weight])
ambi.buildRepresentation(options)        // discretize: mesh/sdf/resolution
ambi.optimize({iterations, method='gradient'|'evolution', callbacks})
ambi.generateWarpMap(viewpointId, mode='homography'|'cylindrical'|'lut')
ambi.renderToTarget(camera, outputFBO)
ambi.evaluateLegibility(cameraOrViewRange) -> {score, mask, pass}
ambi.sampleViewset(bounds, sampler) -> [{v, score}]
ambi.exportMesh(path, format='glb'|'obj')
ambi.exportLookupMap(path)              // exports inverse UV LUT as PNG/EXR
ambi.saveProject(path)
ambi.loadProject(path)

```

Notes on parameters - `cameraMatrix`: 4x4 view matrix or explicit {pos,lookAt,fov,near,far,projection:'persp'|'ortho'}. - `imageOrSDF`: prefer SDF/MSDF for crispness; if image, prefilter into SDF. - `tolerance`: pixel or physical tolerance for legibility.

5. Dataflow / lifecycle

1. **Init**: instantiate AmbiAnamorph with config (resolution limits, optimizer backend, renderer backend).
2. **Load**: `loadSource()` and `setTargets()` define what you want to read from which viewpoint(s).
3. **Build**: `buildRepresentation()` creates an editable mesh / control curves and SDF maps.
4. **Optimize**: `optimize()` runs silhouette & stroke losses + regularizers to shape the geometry such that projections match targets.
5. **Validate**: `evaluateLegibility()` and `sampleViewset()` map `S` and measure robustness.
6. **Finalize**: `generateWarpMap()` to produce `H-1` or `LUT` for runtime warping; `exportMesh()` to get final geometry.
7. **Runtime**: `renderToTarget()` + shader using exported lookup map or direct projection.

6. Losses, metrics & constraints

- **Silhouette loss**: IoU or L2 between rendered mask and target mask (SDF L2 recommended for gradients).
- **Stroke width loss**: match SDF distance band to preserve stroke thickness.
- **Regularizers**: Laplacian mesh smoothing, min-feature thickness, non-self-intersection penalty, volume/weight penalty.
- **Anamorphic objective**: if camera supplies anamorphism, include homography fidelity term between projected texture and target.
- **Legibility metric**: composite of mask-IoU, OCR/confidence (optional), stroke continuity.

7. Implementation choices & recommended defaults

- **Optimizer backend**: gradient: PyTorch3D / TensorFlow Graphics; non-gradient: CMA-ES or CMA-ES on control points.

- **SDF handling:** use MSDF for text; build raster SDF at 1024px by default.
- **Mesh param:** start from coarse (~500-2k verts) and refine.
- **Renderer:** Three.js (WebGL2) for preview and runtime; use an offscreen FBO for intermediate buffers.
- **Warp LUT:** prefer 2× float EXR LUT for high-precision projection mapping; PNG is OK for visual prototypes.

8. Runtime shader snippet (concept)

- **Inputs:** texture `T`, inverse lookup LUT `LUT`, optional Hinv mat.
- **Fragment:** fetch `srcUV = texture(LUT, fragUV)`, sample `T` at `srcUV`.

9. Validation, testing & diagnostics

- **Unit tests:** loader correctness, SDF generator, homography vs LUT equivalence, export/import consistency.
- **Integration tests:** run optimize on small toy cases (two orthographic views) and assert `IoU >= 0.9`.
- **Visual diagnostics:** heatmap of legibility over sampled camera grid, overlay of projected mask vs target, export `S` topology graph.

10. Edge cases & limitations

- Exact multi-view projection solutions do not always exist; the optimizer should return best-effort with quality scores.
- Thin/fragile features may appear in solver results — enforce manufacturability constraints if fabricating.
- Highly nonconvex objectives → multi-stage optimization recommended.

11. Extensibility

- Plugins for new projector types (conical, mirror surfaces), new legibility metrics (ML OCR), and new representation types (volumetric SDF).

12. Deliverables & file outputs

- Optimized mesh (.glb/.obj), lookup LUT (EXR/PNG), SDF/MSDF textures (.png/.exr), project file (.json), diagnostic maps (.png), render target frames (.png/.mp4).

13. Quick example: usage sketch (pseudo)

```
const ambi = new AmbiAnamorph({renderer: 'three', optimizer:
'pytorchdifferentiable'});
await ambi.loadSource({type: 'svg', data: 'wordA.svg'});
ambi.setTargets([{id: 'A', imageOrSDF: 'wordA.sdf'}]);
ambi.setViewpoints([{id: 'vA', cameraMatrix: CAM_A}]);
await ambi.buildRepresentation({mode: 'slats'});
await ambi.optimize({iterations: 500});
ambi.generateWarpMap('vA', 'homography');
```

```
ambi.exportMesh('final.glb');  
ambi.exportLookupMap('warp.exr');
```

Implementation notes for developers

- Carefully separate **offline** heavy work (optimizer) from **runtime** (renderer + LUT apply). Keep the runtime minimal: a mesh, a small LUT texture, and a compact shader.
 - Provide two consumer modes: **Designer** (full optimize/validate) and **Runtime** (fast render + optional viewer tracking).
-

If you want, I can now: (A) produce a Three.js starter repo outline that implements the runtime portion, or (B) provide a PyTorch3D notebook sketch for the offline optimizer. Which would you like?